

Softwarequalität

HOR-TInf2020/1

Gruppenarbeit

Hyperformer_ContactSplitter

Anwendungsdokumentation

12.05.2023

Dipl.-Math. Christian Hübscher

Inhaltsverzeichnis

Hyperformer_ContactSplitter	3
Allgemeine Prinzipien	3
Composite Components und Dependency Inversion	3
Dependency Injection	3
Schichtenarchitektur	3
MVVM	3
CleanCode	3
Anmerkungen zum Parser	4
Konkreter Projektaufbau	4
User Stories	6
Definition Of Done	7
Testfälle	7
Release Note	8

Abbildungsverzeichnis

Abbildung 1: Klassendiagramm Teil eins	9
Abbildung 2: Klassendiagramm Teil zwei	10
Abbildung 3: Klassendiagramm Teil drei	11

Hyperformer_ContactSplitter

Dieses Dokument soll Aufschluss über Architekturentscheidungen geben.

Allgemeine Prinzipien

Im Folgenden allgemeine Architektur-Prinzipien.

Composite Components und Dependency Inversion

Die Composite Components Architektur zielt auf eine Entkopplung von Abhängigkeiten auf Komponentenebene ab. Eine Komponente besteht in .NET immer aus zwei Assemblies: Ein Assembly enthält die Implementierungen, das andere die Schnittstellen und Datenklassen (Contract).

Assemblies besitzen dabei immer nur Abhängigkeiten zu einer Contract Assembly (Dependency Inversion). Dies ermöglicht eine Austauschbarkeit auf Komponentenebene. Dieses Design wird vom Softwarearchitekt David Tilke (<https://www.david-tielke.de>) für .NET-Businessanwendungen empfohlen und orientiert sich an dem ISO-9126 Qualitätsmodell.

Dependency Injection

Das Mapping von Contracts und Implementierungen erfolgt über einen Dependency Injection Container, dessen Konfiguration sich im Hauptprojekt (ContactSplitter) befindet. Das Hauptprojekt konsumiert alle Komponenten und führt entsprechende Kompositionen durch. Diese Schritte erhöhen die Testbarkeit, Wiederverwendbarkeit und Austauschbarkeit der Komponenten und Klassen und erhöhen jeweils die Modularität.

Schichtenarchitektur

Die Schichtenarchitektur sieht vor, dass eine Schicht jeweils nur eine Abhängigkeit zur unmittelbar darunterliegenden Schicht besitzt. Im Projekt gibt es die Schichten UI, Logik und Repository, welche sich in den Assemblies: ContactSplitter.Control (UI), ContactParser.Contracts und ContractParser (Logik) und ContactSplitter.Datastorage (Repository) manifestieren.

MVVM

Die Model-View-ViewModel bezieht sich auf das Zusammenspiel von UI, Daten und UI-Logik. Durch Bindung an ein ViewModel findet die Entkopplung von UI-Design und UI-Logik statt. Somit kann das ViewModel leicht ausgetauscht werden.

CleanCode

Zur Einhaltung von CleanCode und entsprechender Codierungsrichtlinien, wurde neben Sorgfalt der Entwickler, das Werkzeug Resharper verwendet. Dieses Werkzeug führt statische Codeanalysen durch

und kann mittels syntaktischer semantischer Analyse Antipatterns, Inkonsistenzen, mögliche Vereinfachungen sowie Verletzungen von Codierungsrichtlinien (Namenskonventionen, Einrückung, Formatierung...) erkennen. Die verwendeten Codierungsrichtlinien sind die von der Firma JetBrains empfohlenen und im Resharper vorkonfigurierten Richtlinien: <https://www.jetbrains.com/dotnet/guide/tutorials/resharper-essentials/>

All diese Prinzipien sorgen für bessere Wiederverwendbarkeit, Austauschbarkeit, Testbarkeit, Erweiterbarkeit und Modularität (getrennte Entwicklung).

Anmerkungen zum Parser

Für eine Verbesserte Nutzererfahrung gibt es zwei Varianten des Parsers, welche der Nutzer in den Einstellungen auswählen kann.

Offline (Default) Parser: Parst die Nutzereingabe mittels logischer programmatischer Textanalyse. Liefert korrekte Ergebnisse, kann jedoch keine Inferenz mit Weltwissen betreiben (z.B. von dem Namen ‚Hans‘ auf das Geschlecht ‚männlich‘ schließen). Beruht auf der Annahme, dass die Reihenfolge immer „Herr/Frau Titel Vorname Präfix Nachname“ ist. Dabei ist alles außer der Nachname optional und ggf. mehrfach vorhanden. Der Nachname ist immer das letzte zusammenhängende Wort, Doppelnamen müssen also durch „-“ getrennt werden.

Online (GPT) Parser: Verwendet das Sprachmodell GPT3.5. Durch das breite Weltwissen, welches das vortrainierte Natural Language Processing Modell besitzt, kann neben dem Parsen auch logische Inferenz betrieben werden. So kann beispielsweise von dem Namen ‚Hans‘ auf das Geschlecht ‚männlich‘ geschlossen werden. Dieser Parser benötigt eine Internetverbindung und ist nicht streng deterministisch. Die Ergebnisse können bei gleichen Eingaben unterschiedlich sein.

Konkreter Projektaufbau

Projektmappe Hyperformer_ContactSplitter

__ ContactsParser.Contracts	(Siehe Abschnitt "Composite Components")
__ ContactParser	(Logik des klassischen Parsens)
__ GPTContactParser	(Logik des ChatGPT-API Parsers)
__ ContactSplitter.Control	(Wiederverwendbares UI-Element, welches Darstellung, Eingabe, Ausgabe regelt)
__ ContactSplitter	(Hauptprojekt)

__ ContactSplitter.DataStorage.Contracts	(Siehe Abschnitt "Composite Components")
__ ContactSplitter.DataStorage	(Datenhaltung aller Kontakte usw.)
__ ContactSplitter.Tests	(Testprojekt zur Validierung)

Es handelt sich um jeweils einzelne Projekte (Assemblies), welche in einer Projektmappe (Solution) zusammengefasst sind und interagieren. Auf den Seiten 9-11 sind Abbildungen zu sehen, welche die einzelnen Klassendiagramme der Projekte darstellen.

User Stories

- Als Nutzer möchte ich ein manuelles Textfenster haben, um die Zerlegung der Anrede manuell anzupassen, falls die automatische Zuordnung nicht korrekt ist oder ich den Kontakt selbst eintragen möchte.

Akzeptanzkriterien:

- Das System sollte Textfelder zur Verfügung stellen, in dem Nutzer Anreden manuell bearbeiten können.
- Beim Parsen sollen die Textfenster bereits mit den erkannten Personenmerkmalen gefüllt werden

Priorisierung: Hoch

- Als Nutzer möchte ich die Möglichkeit haben, neue Titel manuell hinzuzufügen, um die Genauigkeit der automatischen Zuordnung in Zukunft zu verbessern

Akzeptanzkriterien:

- Das System sollte eine Funktion zum Hinzufügen neuer Titel bieten, die der Nutzer eingeben kann.
- Das System sollte den neu hinzugefügten Titel in der automatischen Zuordnung für zukünftige Fälle berücksichtigen.
- Das System soll die bereits existierenden Titel anzeigen

Priorisierung: Hoch

- Als Nutzer möchte ich, dass das System das Geschlecht einer Person automatisch erkennt, wenn es über die Anrede identifiziert werden kann.

Akzeptanzkriterien:

- Das System sollte das Geschlecht einer Person basierend auf der Anrede automatisch erkennen und zuordnen.

Priorisierung: Mittel

- Als Nutzer möchte ich gespeicherte Kontakte nachträglich verändern und löschen können, um mein Adressbuch aktuell zu halten.

Akzeptanzkriterien:

- Das System sollte es dem Nutzer ermöglichen, gespeicherte Kontaktinformationen zu bearbeiten, einschließlich Anrede, Name, Titel und Geschlecht.
- Das System sollte eine Funktion zum Löschen von Kontakten bereitstellen.
- Änderungen und Löschungen von Kontakten sollten unmittelbar in der Kontaktliste des Nutzers aktualisiert werden.

Priorisierung: Hoch

Definition Of Done

- Alle Anforderungen des Kunden und des Entwicklerteams sind erfüllt und umgesetzt.
- Die App kann die Personenmerkmale in verschiedenen textuellen Repräsentationen korrekt parsen.
- Die App bietet die Möglichkeit bei Bedarf geparte Personenmerkmale manuell anzupassen.
- Die App bietet eine Möglichkeit für den Benutzer, seine Kontakte abzurufen
- Die App bietet eine einfache Möglichkeit für die Benutzer, Personenmerkmale zu suchen und zu filtern.
- Die App bietet die Möglichkeit Kontakte zu löschen
- Die App bietet die Möglichkeit mögliche Titel zu konfigurieren, um künftige Parse-vorgänge zu optimieren.
- Die App ist benutzerfreundlich gestaltet und einfach zu bedienen.
- Die App ist umfänglich dokumentiert, einschließlich der Funktionsweise, der Systemanforderungen und der Architektur.
- Die Dokumentation der App ist für das erste Release angepasst.
- Die App ist erfolgreich getestet und alle gefundenen Fehler und Bugs sind behoben.
- Die App ist bereit für die Veröffentlichung und wird vom Kunden akzeptiert.

Testfälle

Die Testfälle sind im xUnit-Testprojekt „ContactSplitter.Tests“. Sie können von dort abgelesen werden. Alle [InlineData]-Zeilen nach [Theory] stellen einen Testfall dar. Das erste Feld stellt die Eingabe für den Parser dar. Alle weiteren Felder sind die erwarteten Lösungen in folgender Reihenfolge: Anrede, Titel, Briefanrede, Vorname, Nachname, Geschlecht.

Die Testfälle sind möglich breit gestreut:

- Angabe des Geschlechts und keine Angabe
- Kein Titel, ein Titel und mehrere Titel
- Einer und mehrere Nachnamen
- Keiner, einer und mehrere Vornamen
- Abkürzungen als Titel und ausgeschriebene Titel zur Ableitung des Geschlechtes
- Kein oder ein Präfix vor dem Nachnamen

Während beim Default (Offline) Parser alle Testfälle erfolgreich sind, variiert dies beim GPT-Parser. Jede Ausführung kann unterschiedliche Ergebnisse liefern. Pro Parser gibt es bis zu zwanzig Testfälle.

Release Note

Willkommen zur ersten Veröffentlichung der App! Wir freuen uns, Ihnen unsere brandneue Anwendung für die Verwaltung Ihrer Kontakte und Personenmerkmale vorstellen zu können. Die App wurde entwickelt, um Ihnen zu helfen, Ihre Kontakte besser zu organisieren und Informationen schnell zu finden.

Die wichtigsten Funktionen der App in dieser Version sind:

- Die Möglichkeit, Personenmerkmale aus verschiedenen textuellen Repräsentationen zu parsen und zu speichern.
- Eine Funktion, mit der Sie gepasste Personenmerkmale manuell anpassen können, wenn sie falsch oder unzureichend erkannt wurden.
- Eine Such- und Filterfunktion, die es Ihnen ermöglicht, Personenmerkmale einfach zu finden und zu filtern.
- Eine Benutzeroberfläche, die die Bedienung der App einfach und intuitiv nutzbar macht.

Diese erste Veröffentlichung enthält auch einige wichtige Hinweise und Anweisungen für Benutzer:

- Die App ist derzeit nur für Windows Geräte verfügbar.
- Es gibt zwei Varianten der App als ausführbare exe Dateien
 - ContactSplitter 1.0.exe (Beinhaltet die .NET 7.0 Runtime)
 - ContactSplitter 1.0 (.NET 7 Runtime required).exe (Benötigt eine Installation der .NET 7 Runtime: <https://dotnet.microsoft.com/en-us/download/dotnet/7.0>)
- Verwendete Open Source Lizenzen sind in der App auf der Einstellungsseite angegeben
- Die App ist offline nutzbar, benötigt jedoch für eine Spezialfunktion Internetzugriff

Wir hoffen, dass Sie die App genießen werden und freuen uns auf Ihr Feedback, um zukünftige Updates zu verbessern und zu optimieren.

Diese Release Note bietet eine kurze Zusammenfassung der wichtigsten Funktionen der App und enthält wichtige Hinweise und Anweisungen für Benutzer. Sie soll sicherstellen, dass Benutzer die App schnell verstehen und nutzen können.

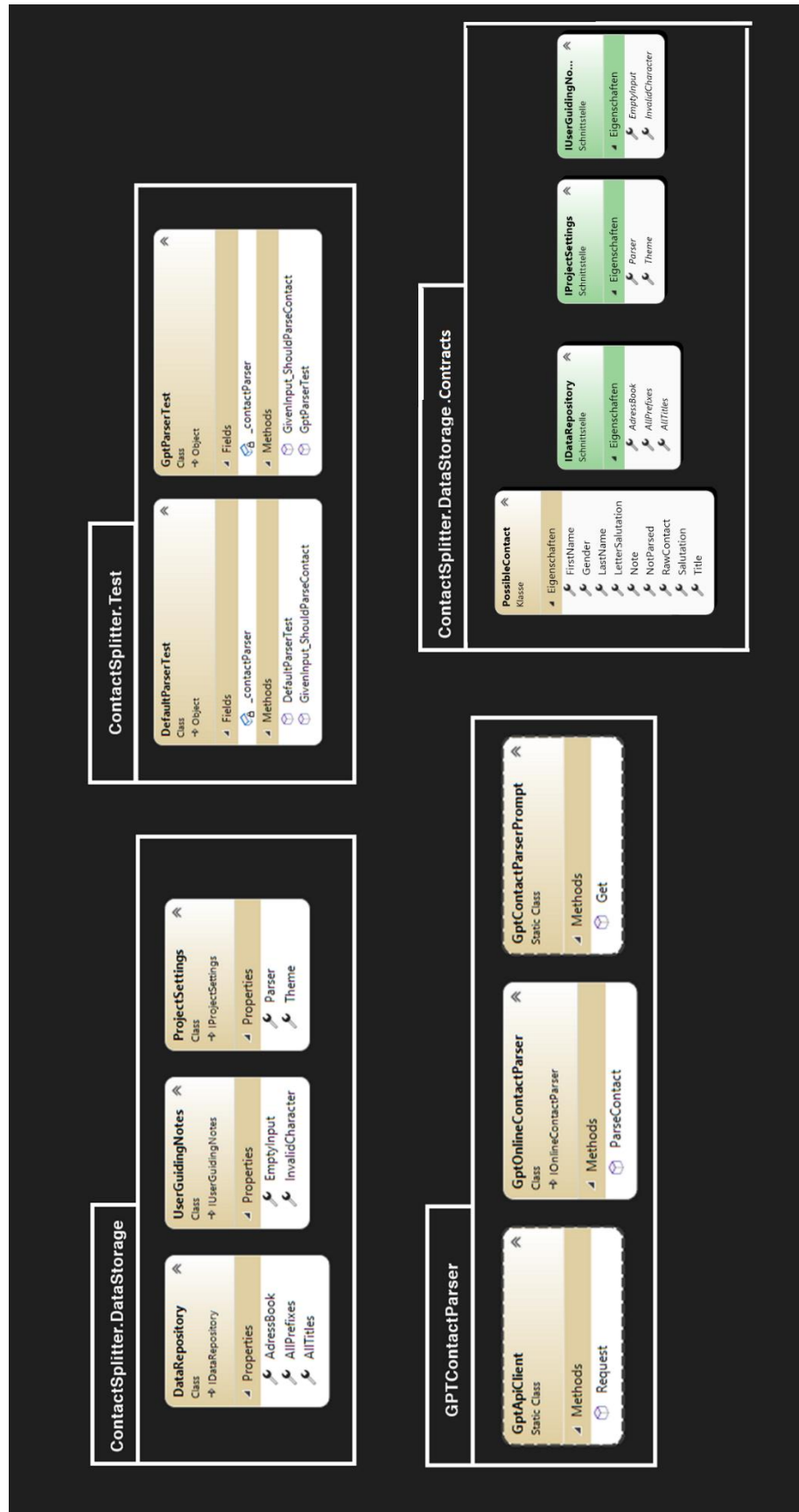


Abbildung 1: Klassendiagramm Teil eins

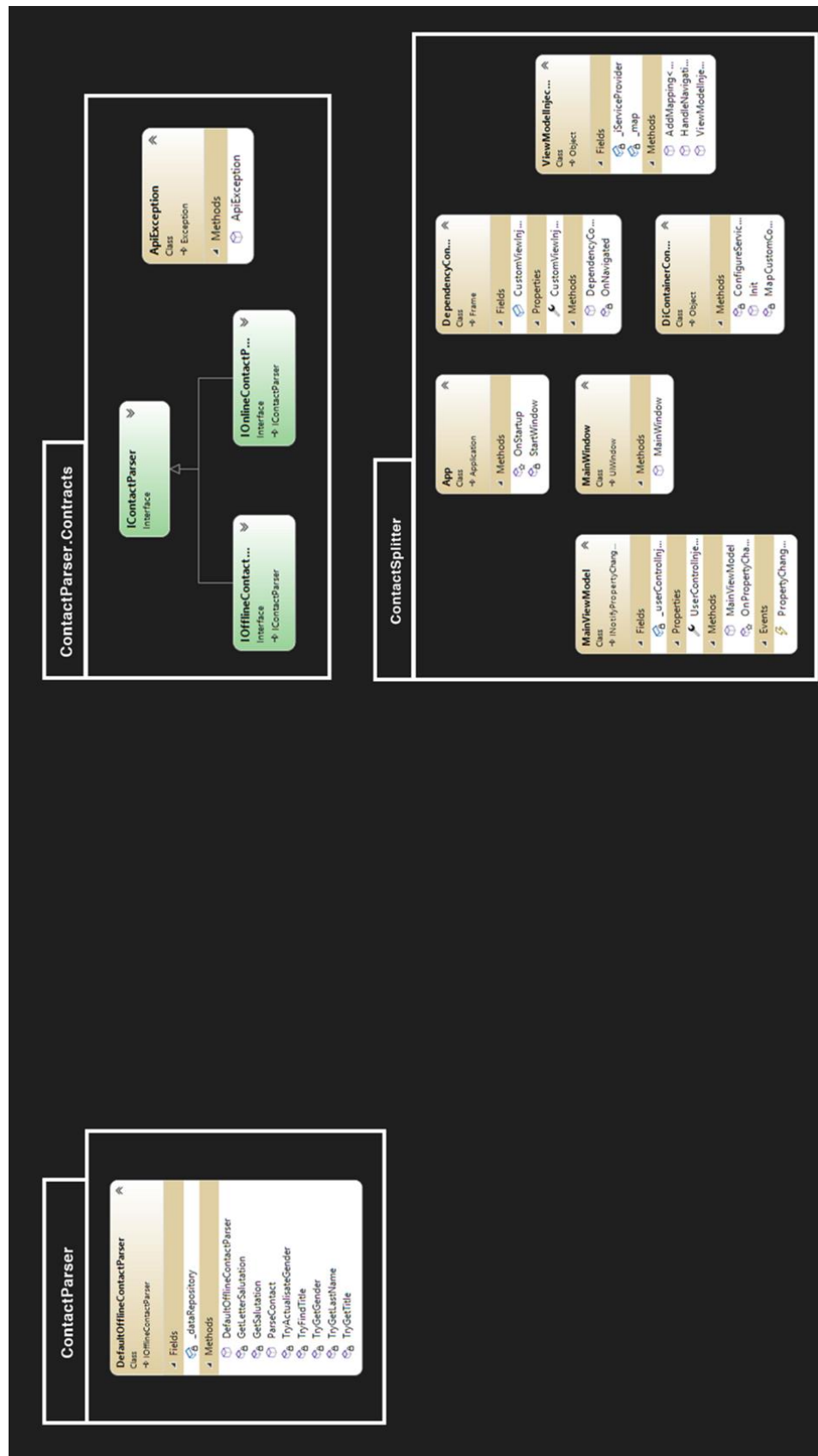


Abbildung 2: Klassendiagramm Teil zwei

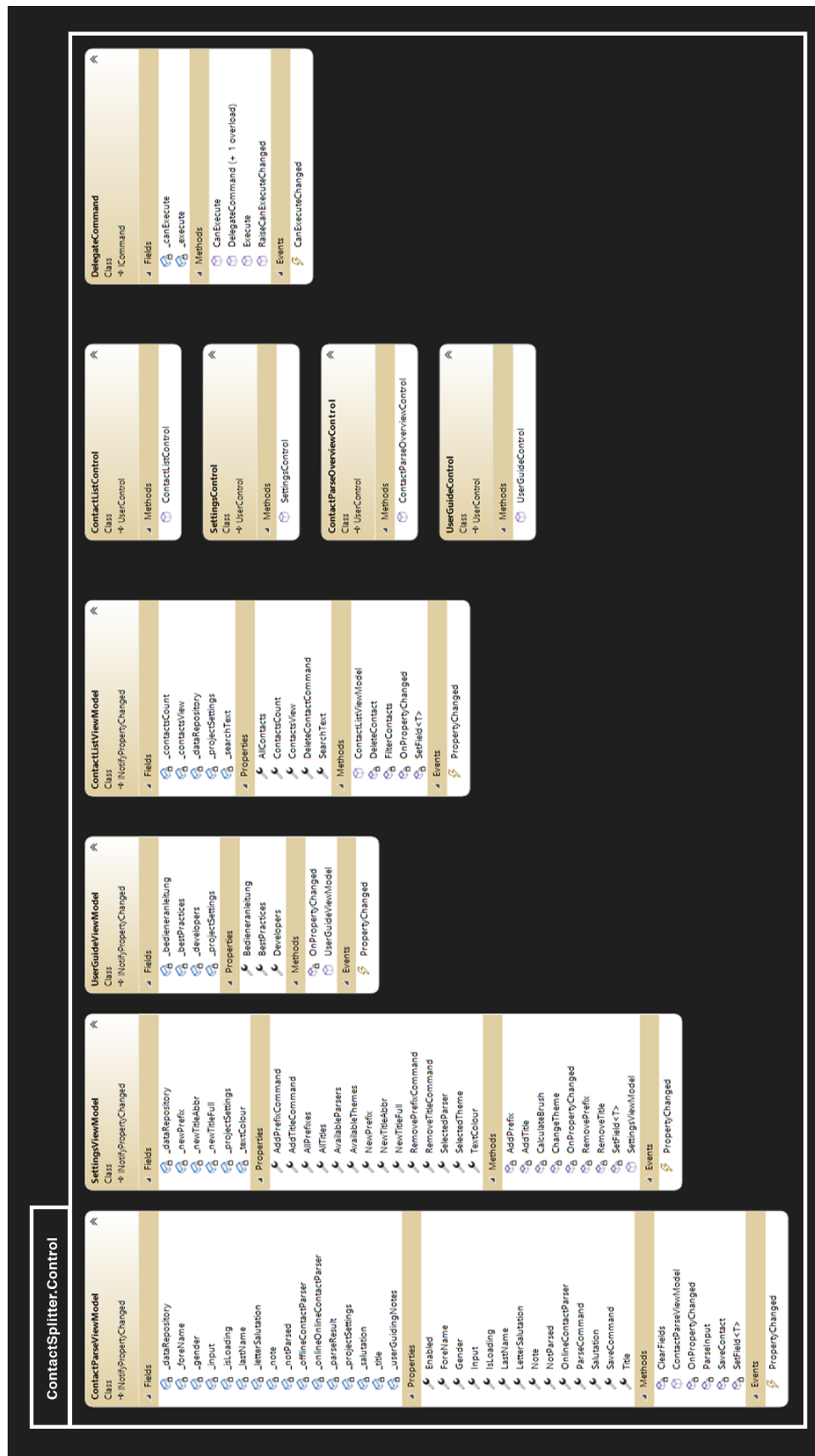


Abbildung 3: Klassendiagramm Teil drei

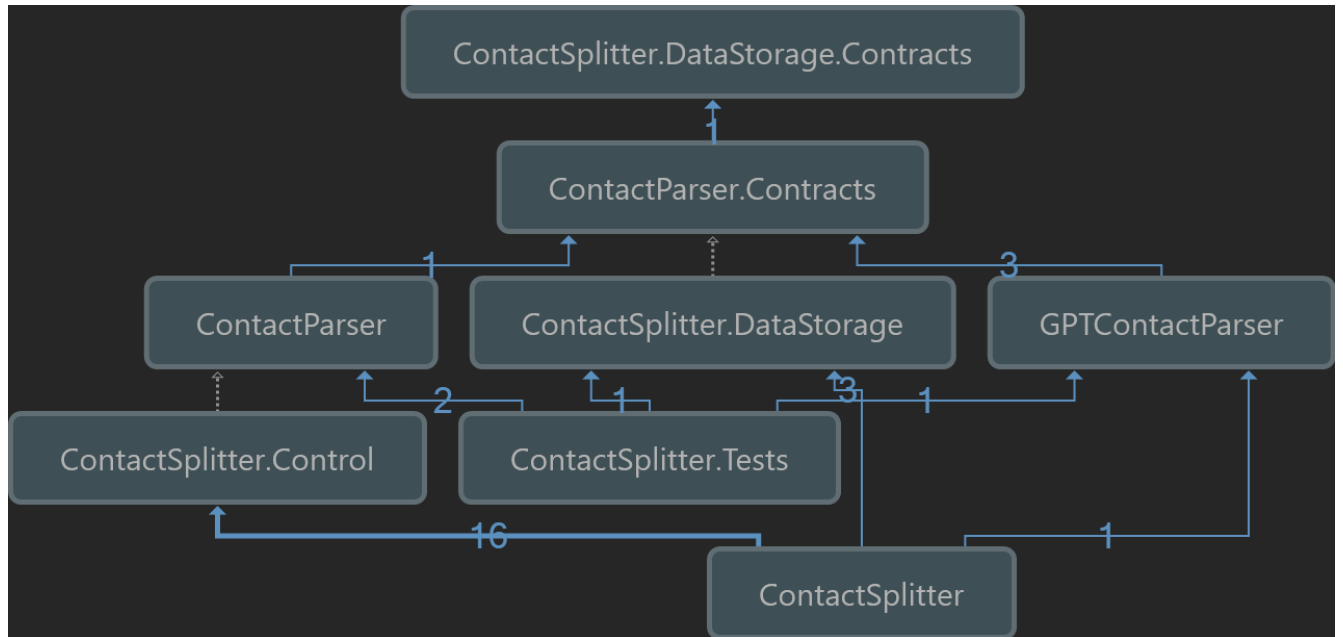


Abbildung 4: Abhängigkeitsdiagramm (Assemblies)